

Sistema de Gestión de Productos y Pedidos

- TechLab

Esta documentación explica paso a paso el funcionamiento del sistema desarrollado en Java. El proyecto utiliza Programación Orientada a Objetos, manejo de excepciones, validaciones y colecciones. Además, incluye espacios marcados para agregar capturas de pantalla posteriormente.

1. Clase Producto

La clase **Producto** representa cada producto del sistema. Contiene atributos como:

- ID
- Nombre
- Precio
- Stock

También incluye:

- Getters y setters
- Método toString() para mostrar información del producto



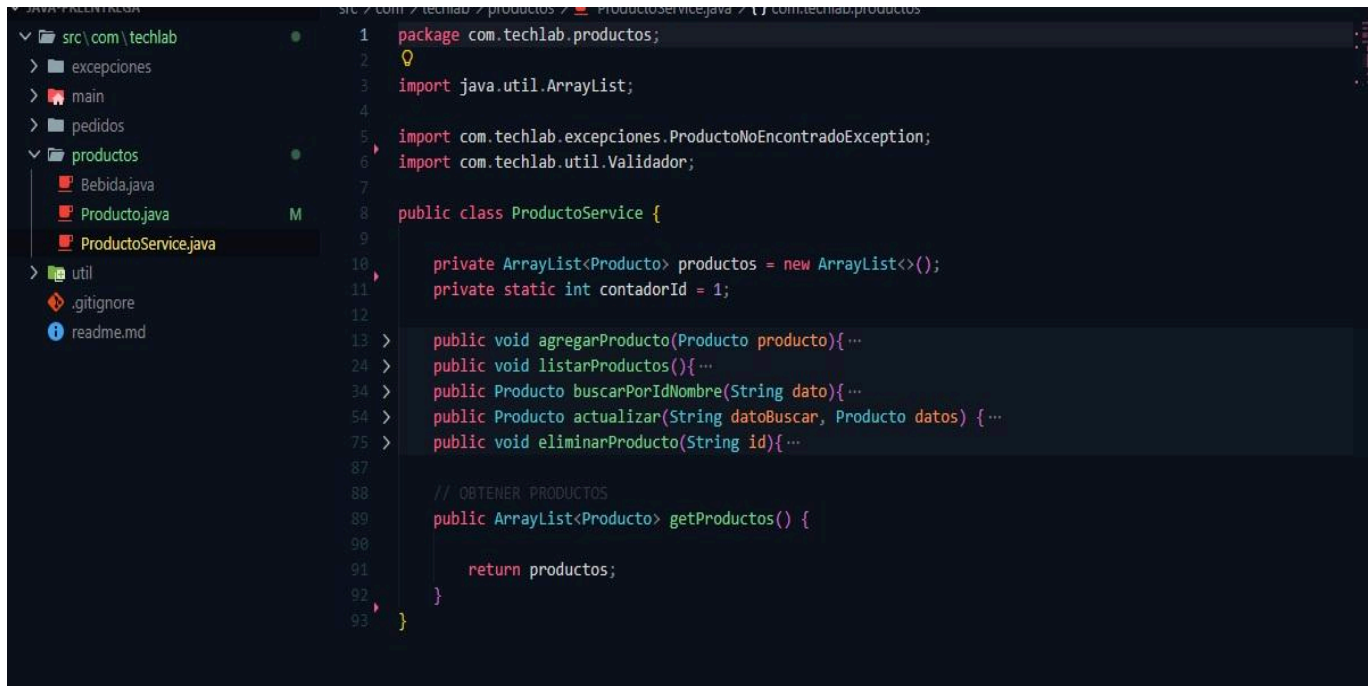
```
1 package com.techlab.productos;
2
3 public class Producto {
4     //ATRIBUTOS
5     private int id;
6     private String nombre;
7     private double precio;
8     private int stock;
9     //CONSTRUCTOR
10    public Producto(String nombre, double precio, int stock) {
11        this.nombre = nombre;
12        this.precio = precio;
13        this.stock = stock;
14    }
15    //GETTERS
16    public int getId() { return id;}
17    public String getNombre() { return nombre;}
18    public double getPrecio() { return precio;}
19    public int getStock() { return stock;}
20    //SETTERS
21    public void setId(int id) { this.id = id;}
22    public void setNombre(String nombre) { this.nombre = nombre;}
23    public void setPrecio(double precio) { this.precio = precio;}
24    public void setStock(int stock) { this.stock = stock;}
25
26    @Override
27    public String toString() {
28        return "ID: " + id + " | Nombre: " + nombre + " | Precio: $" + precio + " | Stock: " + stock;
29    }
30 }
```

2. Clase ProductoService

La clase **ProductoService** administra la lista de productos. Funciones principales:

- Agregar productos
- Listar productos
- Buscar productos por ID o nombre
- Actualizar productos
- Eliminar productos

Se utiliza un `ArrayList` para almacenar los productos en memoria.



```
1 package com.techlab.productos;
2
3 import java.util.ArrayList;
4
5 import com.techlab.excepciones.ProductoNoEncontradoException;
6 import com.techlab.util.Validador;
7
8 public class ProductoService {
9
10     private ArrayList<Producto> productos = new ArrayList<>();
11     private static int contadorId = 1;
12
13     > public void agregarProducto(Producto producto){...
14
15     > public void listarProductos(){...
16
17     > public Producto buscarPorIdNombre(String dato){...
18
19     > public Producto actualizar(String datoBuscar, Producto datos) {...
20
21     > public void eliminarProducto(String id){...
22
23
24     // OBTENER PRODUCTOS
25     public ArrayList<Producto> getProductos() {
26
27         return productos;
28     }
29
30 }
```

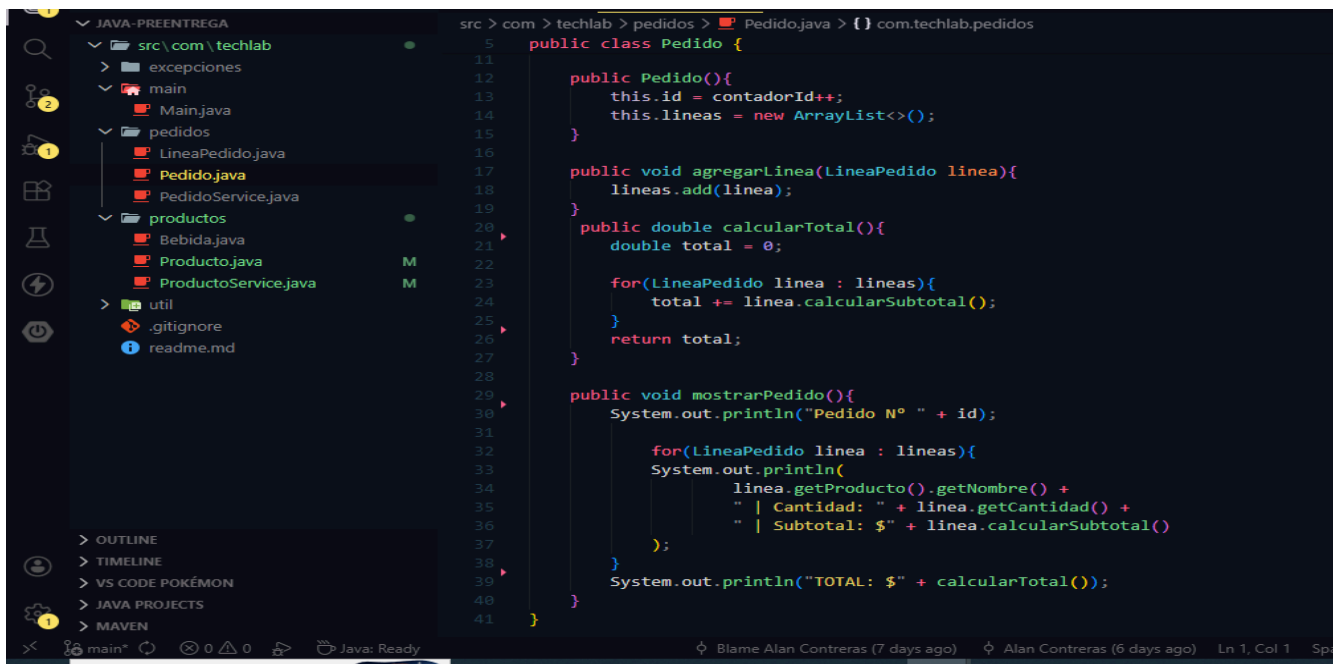
3. Clase Pedido

La clase **Pedido** representa un pedido realizado por el usuario. Cada pedido contiene:

- Un ID autoincremental
- Una lista de líneas de pedido

Además:

- Calcula el total del pedido
- Muestra el resumen del pedido



```
src > com > techlab > pedidos > Pedido.java > {} com.techlab.pedidos

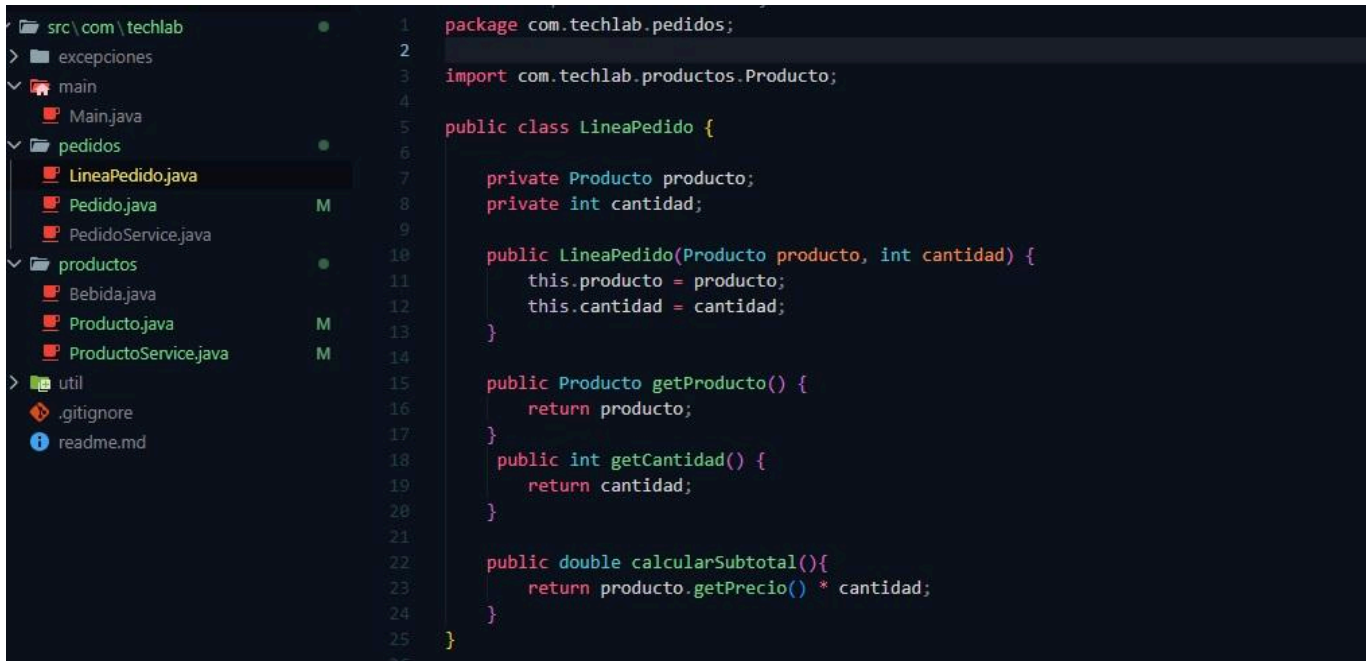
5  public class Pedido {
11
12  public Pedido(){
13      this.id = contadorId++;
14      this.lineas = new ArrayList<>();
15  }
16
17  public void agregarLinea(LineaPedido linea){
18      lineas.add(linea);
19  }
20
21  public double calcularTotal(){
22      double total = 0;
23
24      for(LineaPedido linea : lineas){
25          total += linea.calcularSubtotal();
26      }
27      return total;
28  }
29
30  public void mostrarPedido(){
31      System.out.println("Pedido N° " + id);
32
33      for(LineaPedido linea : lineas){
34          System.out.println(
35              linea.getProducto().getNombre() +
36              " | Cantidad: " + linea.getCantidad() +
37              " | Subtotal: $" + linea.calcularSubtotal()
38          );
39      }
40      System.out.println("TOTAL: $" + calcularTotal());
41  }
```

4. Clase LineaPedido

La clase **LineaPedido** representa un producto dentro de un pedido. Guarda:

- Producto seleccionado
- Cantidad solicitada

También calcula el subtotal multiplicando precio por cantidad.

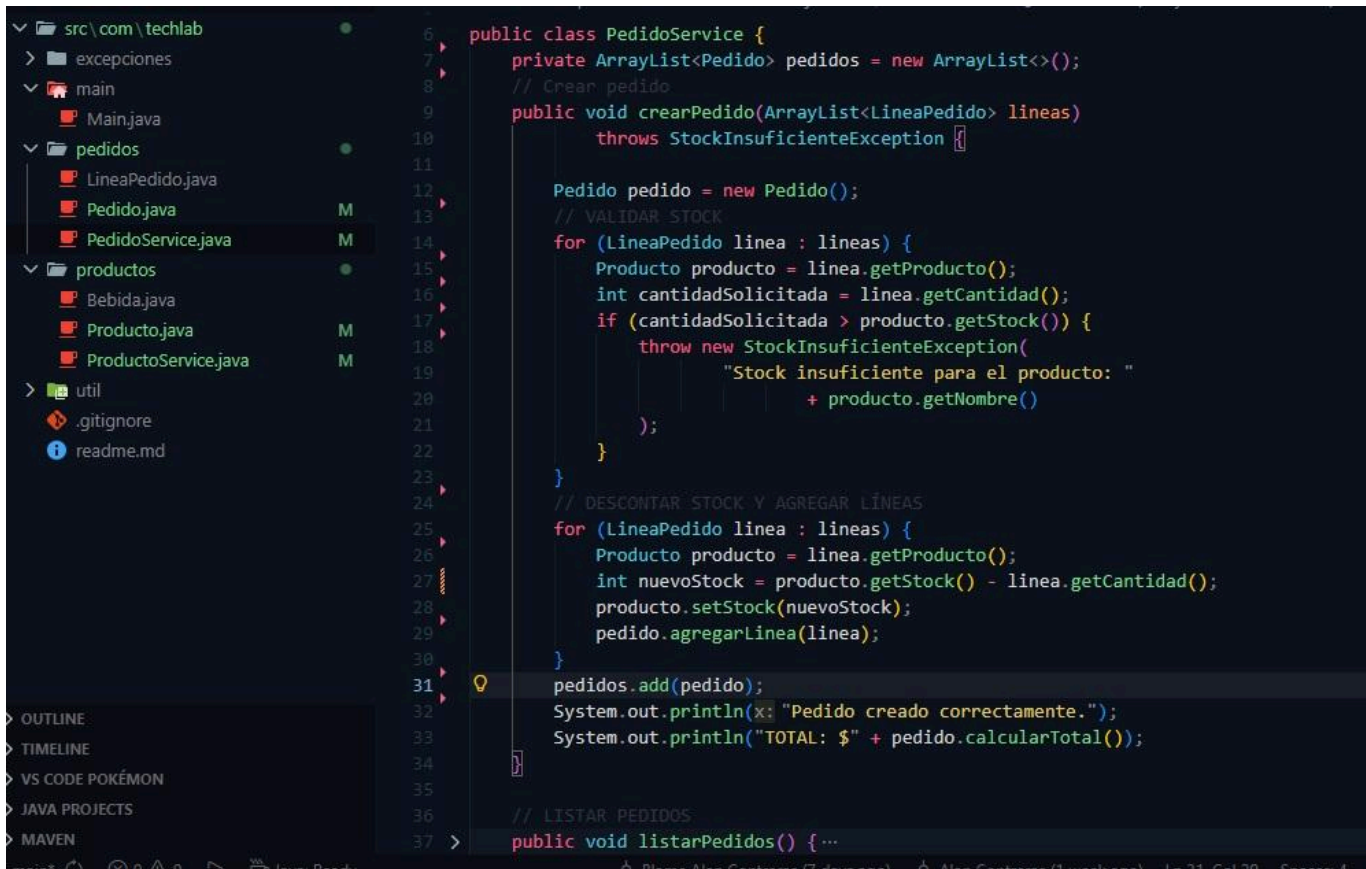


```
1 package com.techlab.pedidos;
2
3 import com.techlab.productos.Producto;
4
5 public class LineaPedido {
6
7     private Producto producto;
8     private int cantidad;
9
10    public LineaPedido(Producto producto, int cantidad) {
11        this.producto = producto;
12        this.cantidad = cantidad;
13    }
14
15    public Producto getProducto() {
16        return producto;
17    }
18    public int getCantidad() {
19        return cantidad;
20    }
21
22    public double calcularSubtotal(){
23        return producto.getPrecio() * cantidad;
24    }
25 }
```

5. Clase PedidoService

La clase **PedidoService** administra los pedidos realizados. Funciones principales:

- Crear pedidos
- Validar stock disponible
- Descontar stock automáticamente
- Guardar pedidos
- Listar pedidos



```
6 public class PedidoService {
7     private ArrayList<Pedido> pedidos = new ArrayList<>();
8     // Crear pedido
9     public void crearPedido(ArrayList<LineaPedido> lineas)
10         throws StockInsuficienteException {
11
12         Pedido pedido = new Pedido();
13         // VALIDAR STOCK
14         for (LineaPedido linea : lineas) {
15             Producto producto = linea.getProducto();
16             int cantidadSolicitada = linea.getCantidad();
17             if (cantidadSolicitada > producto.getStock()) {
18                 throw new StockInsuficienteException(
19                     "Stock insuficiente para el producto: "
20                     + producto.getNombre()
21                 );
22             }
23         }
24         // DESCONTAR STOCK Y AGREGAR LÍNEAS
25         for (LineaPedido linea : lineas) {
26             Producto producto = linea.getProducto();
27             int nuevoStock = producto.getStock() - linea.getCantidad();
28             producto.setStock(nuevoStock);
29             pedido.agregarLinea(linea);
30         }
31         pedidos.add(pedido);
32         System.out.println(x: "Pedido creado correctamente.");
33         System.out.println("TOTAL: $" + pedido.calcularTotal());
34     }
35
36     // LISTAR PEDIDOS
37     public void listarPedidos() { ...
```

6. Clase Validador

La clase **Validador** contiene métodos estáticos reutilizables. Permite:

- Validar nombres
- Validar precios
- Validar stock
- Leer enteros seguros
- Leer doubles seguros

Esto evita errores cuando el usuario ingresa datos inválidos.



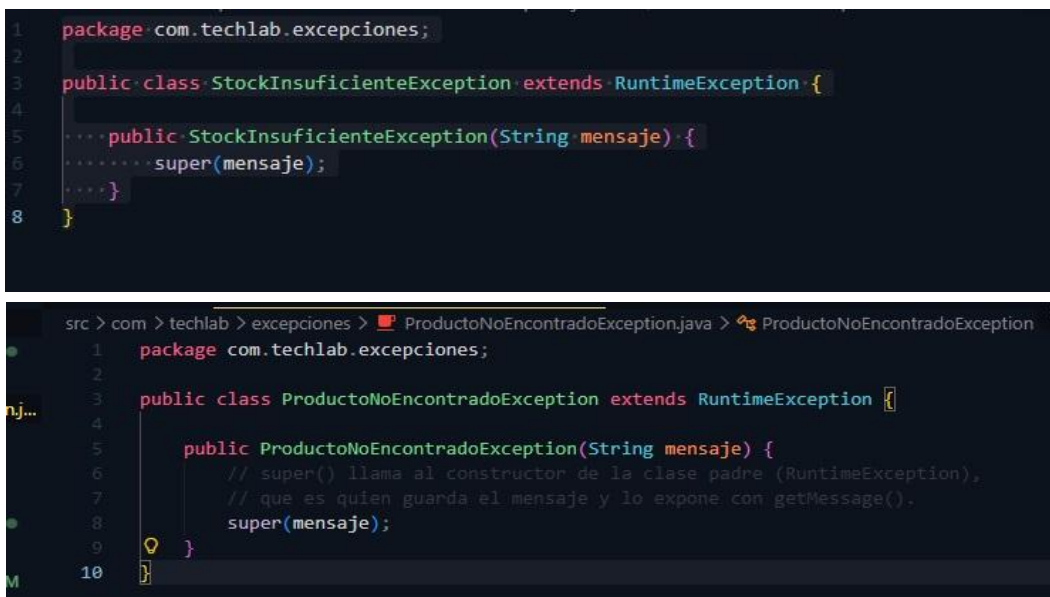
```
1 package com.techlab.util;
2
3 import com.techlab.excepciones.StockInsuficienteException;
4
5 import java.util.InputMismatchException;
6 import java.util.Scanner;
7
8 public class Validador {
9
10 > public static void validarNombre(String nombre) { ...
16
17 > public static void validarPrecio(double precio) { ...
23
24 > public static void validarStock(int stock) { ...
29
30 > public static void validarCategoria(String categoria) { ...
35
36 > public static int leerEntero(Scanner sc, String mensaje) { ...
50
51 > public static double leerDouble(Scanner sc, String mensaje) { ...
64
65 > public static String leerTexto(Scanner sc, String mensaje) { ...
72
73 }
```

7. Excepciones Personalizadas

El proyecto utiliza excepciones personalizadas para mejorar el control de errores.

- ProductoNoEncontradoException
- StockInsuficienteException

Estas excepciones permiten mostrar mensajes claros al usuario.



```
1 package com.techlab.excepciones;
2
3 public class StockInsuficienteException extends RuntimeException {
4
5     ... public StockInsuficienteException(String mensaje) {
6         ... super(mensaje);
7     }
8 }
```

```
src > com > techlab > excepciones > ProductoNoEncontradoException.java > ProductoNoEncontradoException
1 package com.techlab.excepciones;
2
3 public class ProductoNoEncontradoException extends RuntimeException {
4
5     public ProductoNoEncontradoException(String mensaje) {
6         // super() llama al constructor de la clase padre (RuntimeException),
7         // que es quien guarda el mensaje y lo expone con getMessage().
8         super(mensaje);
9     }
10 }
```


8. Clase Main

La clase **Main** contiene el menú principal del sistema. Desde aquí el usuario puede:

- Agregar productos
- Buscar productos
- Actualizar productos
- Eliminar productos
- Crear pedidos
- Listar pedidos

El menú se ejecuta utilizando un bucle do-while.

```
> com > techlab > main > Main.java > Main > main(String[])
14 public class Main {
16     public static void main(String[] args) {
17         System.out.println(x: " SISTEMA DE GESTIÓN - TECHLAB ");
18         System.out.println(x: "=====");
19         System.out.println(x: "1) Agregar producto");
20         System.out.println(x: "2) Listar productos");
21         System.out.println(x: "3) Buscar producto");
22         System.out.println(x: "4) Actualizar producto");
23         System.out.println(x: "5) Eliminar producto");
24         System.out.println(x: "6) Crear pedido");
25         System.out.println(x: "7) Listar pedidos");
26         System.out.println(x: "8) Salir");
27
28         opcion = Validador.leerEntero(scanner, mensaje: "Elija una opción: ");
29
30         switch (opcion) {
31             case 1: ...
32             case 2: ...
33             case 3: ...
34             case 4: ...
35             case 5: ...
36             case 6: ...
37             case 7: ...
38             case 8: ...
39             default: ...
40         }
41
42         while (opcion != 8);
43         scanner.close();
44     };
45 }
46
```

Blame Alan Contreras (7 days ago) Alan Contreras (1 week ago) Ln 30, Col 2 Spaces: 4 UTF-8 CRLF {

Conclusión

Este proyecto permite practicar conceptos fundamentales de Java, Programación Orientada a Objetos y manejo de excepciones. Además, la estructura modular facilita futuras mejoras como:

- Persistencia con bases de datos
- API REST
- Interfaz gráfica
- Sistema de usuarios
- Reportes PDF

MENU DEL SISTEMA

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Estudiante\Desktop\Java-PreEntrega> & 'C:\Program Files\java\jdk-17.0.0.1\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Estudiante\AppData\Roaming\Code\User\workspaceStorage\208b1be57857ddd85291641dfe157ef6\redhat.java\jdt_ws\Java-PreEntrega_e8fc6d2f\bin' 'com.techlab.main.Main'

=====
SISTEMA DE GESTIÓN - TECHLAB
=====
1) Agregar producto
2) Listar productos
3) Buscar producto
4) Actualizar producto
5) Eliminar producto
6) Crear pedido
7) Listar pedidos
8) Salir
Elija una opción: 
```

EJEMPLO DE PRODUCTO

```
PS C:\Users\Estudiante\Desktop\Java-PreEntrega> & 'C:\Program Files\java\jdk-17.0.0.1\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Estudiante\AppData\Roaming\Code\User\workspaceStorage\208b1be57857ddd85291641dfe157ef6\redhat.java\jdt_ws\Java-PreEntrega_e8fc6d2f\bin' 'com.techlab.main.Main'

=====
1) Agregar producto
2) Listar productos
3) Buscar producto
4) Actualizar producto
5) Eliminar producto
6) Crear pedido
7) Listar pedidos
8) Salir
Elija una opción: 1
Nombre del producto: CAFE PREMIUM
Precio: 9000
Stock: 70
Producto agregado correctamente.

=====
SISTEMA DE GESTIÓN - TECHLAB
=====
1) Agregar producto
2) Listar productos
3) Buscar producto
4) Actualizar producto
5) Eliminar producto
6) Crear pedido
7) Listar pedidos
8) Salir
Elija una opción: 2
ID: 1 | Nombre: CAFE PREMIUM | Precio: $9000.0 | Stock: 70
```


EJEMPLO DE PEDIDO

```
PS C:\Users\Estudiante\Desktop\Java-PreEntrega> & 'C:\Program Files\java\jdk-17.0.0.1\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Estudiante\AppData\Roaming\Code\User\workspaceStorage\208b1be57857ddd85291641dfe157ef6\redhat.java\jdt_ws\Java-PreEntrega_e8fc6d2f\bin' 'com.techlab.main.Main'

1) Agregar producto
2) Listar productos
3) Buscar producto
4) Actualizar producto
5) Eliminar producto
6) Crear pedido
7) Listar pedidos
8) Salir
Elija una opción: 6
Ingrese ID ó Nombre del producto: 1
Cantidad: 10
¿Agregar otro producto? (s/n): N
Pedido creado correctamente.
TOTAL: $90000.0

=====
SISTEMA DE GESTIÓN - TECHLAB
=====

1) Agregar producto
2) Listar productos
3) Buscar producto
4) Actualizar producto
5) Eliminar producto
6) Crear pedido
7) Listar pedidos
8) Salir
Elija una opción: 7
Pedido N° 1
CAFE PREMIUM | Cantidad: 10 | Subtotal: $90000.0
TOTAL: $90000.0
```

```
PS C:\Users\Estudiante\Desktop\Java-PreEntrega> & 'C:\Program Files\java\jdk-17.0.0.1\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Estudiante\AppData\Roaming\Code\User\workspaceStorage\208b1be57857ddd85291641dfe157ef6\redhat.java\jdt_ws\Java-PreEntrega_e8fc6d2f\bin' 'com.techlab.main.Main'
```

Ingrese ID ó Nombre del producto: 2

Cantidad: 5

¿Agregar otro producto? (s/n): S

Ingrese ID ó Nombre del producto: 3

Cantidad: 3

¿Agregar otro producto? (s/n): N

Pedido creado correctamente.

TOTAL: \$53500.0

```
=====
SISTEMA DE GESTIÓN - TECHLAB
=====
```

- 1) Agregar producto
- 2) Listar productos
- 3) Buscar producto
- 4) Actualizar producto
- 5) Eliminar producto
- 6) Crear pedido
- 7) Listar pedidos
- 8) Salir

Elija una opción: 7

Pedido N° 1

CAFE PREMIUM | Cantidad: 10 | Subtotal: \$90000.0

TOTAL: \$90000.0

Pedido N° 2

HIERBA 1 KILO | Cantidad: 5 | Subtotal: \$17500.0

FERNET | Cantidad: 3 | Subtotal: \$36000.0

TOTAL: \$53500.0